

Experiment 'p_4_1_c_powers_of_two_seq' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_4_1_c_powers_of_two_seq

Natural language statement: Write a method with loops that computes all powers of 2 from 2^0 up to 2^{20} .

Method signature: p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)

Ensures

- `|powers| == 21`
- forall `i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)`

Functional Code Given

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method with loops that computes all powers of 2 from 2^0 up to 2^{20} .

The signature should be:

`p_4_1_c_powers_of_two_seq()` returns `(powers: seq<int>)`

The method should respect the following contract:

ensures `|powers| == 21`, ensures forall `i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)`

The contract uses the following dafny code:

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdzQm4Gm323ZyAntEVP5DZFONwpe

Model: gpt-5.5-2026-04-23

Created at: 1780485836

Finish reason: stop

Usage: 468 tokens in, and 6615 tokens out

```
{
  powers := [];
  var i := 0;
  var current := 1;

  while i < 21
    invariant 0 <= i <= 21
    invariant |powers| == i
    invariant current == pow(2, i)
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
```

```

    decreases 21 - i
  {
    var nextI := i + 1;
    ghost var oldPowers := powers;

    powers := oldPowers + [current];
    assert |powers| == nextI;
    assert powers[i] == current;

    forall j | 0 <= j < nextI
      ensures powers[j] == pow(2, j)
    {
      if j < i {
        assert powers[j] == oldPowers[j];
      } else {
        assert j == i;
        assert powers[j] == current;
      }
    }

    current := current * 2;
    assert current == pow(2, i) * 2;
    assert current == 2 * pow(2, i);
    assert 0 < nextI;
    assert nextI - 1 == i;
    assert pow(2, nextI) == 2 * pow(2, nextI - 1);
    assert pow(2, nextI) == 2 * pow(2, i);
    assert current == pow(2, nextI);

    i := nextI;
  }

  assert i == 21;
  assert |powers| == 21;
  forall j | 0 <= j < |powers|
    ensures powers[j] == pow(2, j)
  {
    assert j < i;
  }
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_4_1_c_powers_of_two_seq/program.dfy(26,14): Error: ghost variables such as
  → oldPowers are allowed only in specification contexts. oldPowers was inferred to
  → be ghost based on its declaration or initialization.
|
26 | powers := oldPowers + [current];
|   ~~~~~

```

1 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
↪ Dafny body, including the curly braces that surround it. Do not show the signature
↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dme1TAGhRbg9B2VUwXpiiABStdXRX

Model: gpt-5.5-2026-04-23

Created at: 1780485963

Finish reason: stop

Usage: 980 tokens in, and 890 tokens out

```
{
  powers := [];
  var i := 0;
  var current := 1;

  while i < 21
    invariant 0 <= i <= 21
    invariant |powers| == i
    invariant current == pow(2, i)
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
    decreases 21 - i
  {
    var nextI := i + 1;
    var oldPowers := powers;

    powers := oldPowers + [current];
    assert |powers| == nextI;
    assert powers[i] == current;

    forall j | 0 <= j < nextI
      ensures powers[j] == pow(2, j)
    {
      if j < i {
        assert powers[j] == oldPowers[j];
      } else {
        assert j == i;
        assert powers[j] == current;
      }
    }
  }

  current := current * 2;
  assert current == pow(2, i) * 2;
  assert current == 2 * pow(2, i);
  assert 0 < nextI;
  assert nextI - 1 == i;
```

```

    assert pow(2, nextI) == 2 * pow(2, nextI - 1);
    assert pow(2, nextI) == 2 * pow(2, i);
    assert current == pow(2, nextI);

    i := nextI;
}

assert i == 21;
assert |powers| == 21;
forall j | 0 <= j < |powers|
    ensures powers[j] == pow(2, j)
{
    assert j < i;
}
}

```

Final Program

```

function pow(base: int, exp: int): int
    requires 0 <= exp
    decreases exp
{
    if exp == 0 then 1 else base * pow(base, exp - 1)
}

method p_4_1_c_powers_of_two_seq() returns (powers: seq<int>)
    ensures |powers| == 21
    ensures forall i :: 0 <= i < |powers| ==> powers[i] == pow(2, i)
{
    powers := [];
    var i := 0;
    var current := 1;

    while i < 21
        invariant 0 <= i <= 21
        invariant |powers| == i
        invariant current == pow(2, i)
        invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
        decreases 21 - i
    {
        var nextI := i + 1;
        var oldPowers := powers;

        powers := oldPowers + [current];
        assert |powers| == nextI;
        assert powers[i] == current;

        forall j | 0 <= j < nextI
            ensures powers[j] == pow(2, j)
        {

```

```

    if j < i {
        assert powers[j] == oldPowers[j];
    } else {
        assert j == i;
        assert powers[j] == current;
    }
}

current := current * 2;
assert current == pow(2, i) * 2;
assert current == 2 * pow(2, i);
assert 0 < nextI;
assert nextI - 1 == i;
assert pow(2, nextI) == 2 * pow(2, nextI - 1);
assert pow(2, nextI) == 2 * pow(2, i);
assert current == pow(2, nextI);

i := nextI;
}

assert i == 21;
assert |powers| == 21;
forall j | 0 <= j < |powers|
    ensures powers[j] == pow(2, j)
{
    assert j < i;
}
}

```

Total Token Usage

Input tokens: 1448

Output tokens: 7505

Reasoning tokens: 6590

Sum of ‘total tokens’: 8953

Experiment Timings

Overall Experiment started at 1780485835810, ended at 1780485998124, lasting 162314ms (162.31 seconds)

Iteration #1 started at 1780485835811, ended at 1780485962951, lasting 127140ms (127.14 seconds)

Iteration #2 started at 1780485962956, ended at 1780485998123, lasting 35167ms (35.17 seconds)